# EPSR-test-ex72 sharm: nsumt = 5 without a previous file, orientation test; mapgr, mapgrentropy

Program(s): sharm, mapgr, mapgrentropy | chained passes: 2 | verification: ALL IDENTICAL (all 2 passes)

## What this case tests

Covers special paths of `sharm` [1]: accumulation requested with no previous coefficient file, an `rmax` larger than half the box, and the built-in orientation test (`test` keyword) that evaluates the expansion for fixed molecular orientations; the test coefficients are then fed into `mapgrentropy`.

## Code exercised

**Fortran (original, fortran_src/):**

- `sharm_rev2.f`: `run_sharm` (keywords, `nsumt` rule), `sharm` (centres, axes, pair loop, averaged molecules)
- `sharm_routines_rev1.f`: `get_l_and_n_values`, `form_cgcoeffs`, `form_eulerangles`, `form_dlmn`, `form_sum_prod_dlmn_cgcoeff`
- `auxiliary_routines.f`: `dlm`, `cgcoef`, `rotateeuler`; `read_and_save_files.f`: `outgofzs_sharm` (`.h0n` files, averaging); `process_symbols.f`: `check_symbol`
- `mapgrquad.f`: `run_mapgr`; `plotsharm_routines.f`: `smoosharm`, `sumoverm2`; `plot_routines.f`: `find_plot_par`
- `mapgrentropy.f`: `run_mapgrentropy` (entropy sums); `auxiliary_routines.f`: `initran1`, `saveran1`

**Rust (port, rust/epsr-core/src/):**

- `tools/sharm.rs`: `run`, `sharm`, `read_axis`, `read_l_values`, `average_molecule`
- `tools/sharm_routines.rs`: `Sharm` methods with the Fortran names (incl. `outgofzs_sharm`), `dlm`, `cgcoef`, `form_eulerangles`, `rotateeuler`
- `tools/mapgr.rs`: `run`; `tools/sharm_routines.rs`: `Coeffs::read`, `Coeffs::angular`, `smoosharm`, `sumoverm2`, `check_symbol_mapgr`; `output.rs`: `w_int`
- `tools/mapgrentropy.rs`: `run`, `initran1`; `rng.rs`: `Ran1::ran1`

## Physical system

The EPSR-test configuration `200DMSO400Water.ato`: 200 DMSO (atoms S, C, C, O, 6 H) and 400 water molecules (O1, H1, H1; SPC/E parameters [2]), 3200 atoms at 300 K in a cubic box of side 33.12 Å, a model of the DMSO-water mixtures of Soper and Luzar [3]. `sharm` expands the orientational pair correlation function of a central and a second molecule type in rotation matrices coupled by Clebsch-Gordan coefficients; the coefficients g(l1,l2,l;n1,n2;r) are averages over all molecule pairs [4,5]. `watdmso`: central water, second DMSO (DMSO occurs first in the `.ato`, so the "molecule 2 at origin" pass is used), axes z,x and -z,x, l2 = 0 2 3, l = 0-5, n1step 2, n2step 1 (127 coefficients), `averagemol n`. `orient`: `test 10 20 30 40 50 60 70 80` fixes phi/theta/chi of both molecules and the lab vector, axes y,z and y,-z (126 coefficients). `mapgr` rebuilds g(r) along chosen directions and relative orientations from the coefficients (grtype 1 radial, 2 spatial, 3 full orientational). `mapgrentropy` adds to the ideal-gas entropy the two-body excess term, an integral of $r^2 \cdot (g - 1 - g \ln g)$ [6], for the radial, spatial and full g, sampling random orientations with `ran1` [7].

## Inputs

Input files written by hand from the keyword tables (`setup_symbols_*`), one `keyword value` per line, ended by `q`; `200DMSO400Water.ato` copied from EPSR-test. `watdmso.SHARM.dat`: `rmax 30`, `nsumt 5`; `orient.SHARM.dat`: extra blanks in `atom-s` (reformatted by `check_symbol`); `mapgr` views grtype 3 and 2; `orient.mapgrentropy.dat`: `ident -1`, `mirrorsym y`, smoothing `1.0 0 1 2`.

## How the accuracy was verified

`commands.txt` (4 program calls, repeated 2 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. A run-time FORMAT in the original source (`sharm_rev2.f`/`sdf_rev2.f`, and `sharm_diffuse` for `epsr`) lacks a comma after the Hollerith `1h(` and is rejected by current gfortran, so the reference build inserts that comma (originals kept as `*.orig`); the port behaves like the patched program. Result of the final regression run of the complete port: every file identical in all 2 passes.

Two passes. Pass 1: `outgofzs_sharm` fails to read the missing `.h01` (`nsum = -1`, no averaging); `rmax` is capped to half the box (r step 0.276 Å in `watdmso.mapgr.g01`); pass 2 (`nsumt 6`) skips the calculation, final `nsumt 7`. The orientation test does not increment `nsumt` (0 in the reference), so pass 2 recomputes it. Reference build: `sharm`/`sdf` were rebuilt with one comma inserted in the run-time title format (`'(1h(,'//il1fmt`), since gfortran 15.2 otherwise stops with "Missing comma between descriptors" before writing the coefficients; all other programs are the unmodified reference binaries. Also masked: characters 31-45 of the `*.mapgr.g01` data lines, the error column of `g_centres(r)`, which `mapgrquad.f` never sets (uninitialised in the Fortran, 0 in the port).

## References


[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] H. J. C. Berendsen, J. R. Grigera and T. P. Straatsma, "The missing term in effective pair potentials", J. Phys. Chem. 91, 6269-6271 (1987).
[3] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[4] A. K. Soper, "Orientational correlation function for molecular liquids: The case of liquid water", J. Chem. Phys. 101, 6888-6901 (1994).
[5] C. G. Gray and K. E. Gubbins, Theory of Molecular Fluids, Vol. 1: Fundamentals, Clarendon Press, Oxford (1984).
[6] A. Baranyai and D. J. Evans, "Direct entropy calculation from computer simulation of liquids", Phys. Rev. A 40, 3817-3822 (1989).
[7] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[8] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.